Lab 5 Report: Monte-Carlo Localization Using LiDAR, Odometry, and Particle Filtering

Team 2

Nina Garcia-Ruiz
Brendon Murphy
Irene Tang
Vandell Vatel

6.4210 Robotics: Science and Systems

April 10, 2026

1 Introduction

Autonomous mobile robots require reliable knowledge of their own position within a known environment — a problem known as *localization*. In this lab, we designed and implemented a Monte-Carlo Localization (MCL) system for a miniature race car operating in a known 2D map. The system uses two sources of sensor information: wheel odometry, which tracks how the car has moved over time, and LiDAR scan data, which measures the distances to surrounding obstacles. Together, these allow the robot to maintain a probabilistic belief state over its pose (position and orientation) in the map.

MCL is a particle filter-based algorithm: it represents the belief as a cloud of weighted particles, each embedding a hypothesis about where the robot might be. Patricles are translated throught the map via the odometry and in general, particles inconsistent with sensor observations are discarded in favor or more likely ones, causing the cloud to converge to a true pose.

This lab is a foundational step toward a fully autonomous car on our race car model. Accurate localization is prerequisite for many higher-level autonomous functions, such as path planning: without knowing where it is, the car cannot navigate to where it needs to go. Our implementation consists of three modules - a motion model (handles odometry data), a sensor model (handles LiDAR data), and a particle filter (fuses the two models) - all implemented in Python using ROS2. We evaluated the system in both a simulator and on the physical car in the MIT Stata basement.

2 Technical Approach

2.1 Overview

Monte-Carlo Localization maintains a set of N particles $\mathbf{x} = \{x^{(1)}, \ldots, x^{(N)}\}$, where each particle $x^{(i)} = [x, y, \theta]^\top$ represents a candidate pose of the robot. At each timestep (determined by the frequency of each sensor), the filter performs two steps: (1) it propagates particles forward using the motion model given new odometry data, (2) it reweighs particles and assigns each one likelihoods given new LiDAR scans. Particles are then resampled proportional to their weights to maintain a fixed number of N particles. Finally, the robot's estimated pose is take as the circular mean of the particle distribution.

2.2 Motion Model

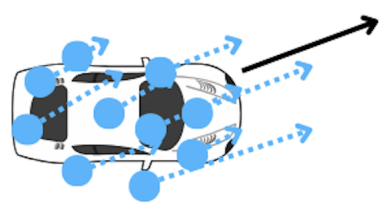


Figure 1: Diagram visualizing the motion model. The blue dots represent the particles that are hypotheses of the robot's position. The black arrow represents the current odometry of the car. The blue dashed arrows represent the odometry being applied to each particle, updating each one to reflect the current kinematics.

The motion model updates previous particle poses $\mathbf{x}_{k-1}$ using noisy odometry data $\mathbf{u}_k = [\Delta x, \Delta y, \Delta\theta]^\top$ to predict the robot's new state $\mathbf{x}_k$. By simulating the robot's kinematics, particles track the car's physical trajectory:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k + \epsilon_u) \tag{1}$$

where f defines the deterministic kinematics and ϵ_u represents odometry noise.

2.2.1 Update Step

To apply the deterministic update f, we separate the pose and odometry into position and orientation components. We transform the local odometry position into the global frame using the rotation matrix $\mathbf{R}(\theta_{k-1})$ and add the orientation change directly:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) = \begin{bmatrix} \mathbf{x}_{k-1,\text{pos}} + \mathbf{R}(\theta_{k-1})\mathbf{u}_{k,\text{pos}} \\ \mathbf{x}_{k-1,\theta} + \mathbf{u}_{k,\theta} \end{bmatrix} \tag{2}$$

where $\mathbf{x}_{k,\text{pos}} = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$, $\mathbf{u}_{k,\text{pos}} = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$ $\mathbf{x}_{k,\theta} = \theta_k$, and $\mathbf{u}_{k,\theta} = \Delta\theta$.

2.2.2 Noising Step

Real-world kinematics suffer from cross-coupled errors in translation and rotation. for instance, rotating causes translational slip, and translating causes rotational drift. To model this, we sample noise $\epsilon_u = [\epsilon_x, \epsilon_y, \epsilon_\theta]^\top$ from zero-mean Gaussians: $\mathcal{N}(0, \sigma_x^2)$, $\mathcal{N}(0, \sigma_y^2)$, and $\mathcal{N}(0, \sigma_\theta^2)$.
These standard deviations scale based on the variance of the odometry data and user-defined α coefficients:

$$\sigma_x = \sqrt{\alpha_1(\Delta x)^2 + \alpha_2(\Delta\theta)^2}$$
$$\sigma_y = \sqrt{\alpha_1(\Delta y)^2 + \alpha_2(\Delta\theta)^2}$$
$$\sigma_\theta = \sqrt{\alpha_3(\Delta x)^2 + \alpha_4(\Delta\theta)^2}$$

Here, α_1 and α_4 scale independent noise (effect of position and orientation change on translational and rotational noise respectively), while α_2 and α_3 govern the cross-coupled noise (effect of position change on rotational noise and effect of orientation change on translational noise).

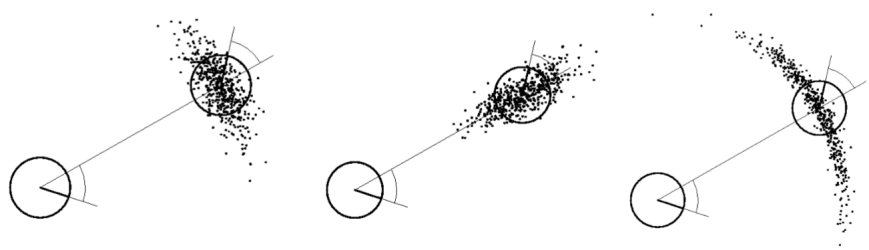


Figure 2: Representations of how the noise term affects the particles' spread given the robot's odometry. The first example occurs when there is similar position and orientation change, the second example occurs when there is large position change compared to orientation, and the third example occurs when there is large orientation change compared to position.

2.2.3 Optimizations

We implemented two modifications to improve real-world performance:

- **Midpoint Integration:** Instead of rotating $\mathbf{u}_k$ by the initial heading θ_{k-1}, we use the average heading $\theta_{\text{avg}} = \theta_{k-1} + \frac{\Delta\theta}{2}$. This better approximates continuous curved motion rather than instantaneous point-turns at each time step.
- **Forward Bias:** To counteract system latency that causes particles to artificially lag behind the ground-truth car, we scale Δx by a multiplier $\alpha_{\text{forward}} > 1.0$, pushing the particle cloud slightly ahead to better match where they should be.

2.3 Sensor Model

The sensor model assigns a likelihood to each particle by comparing the LiDAR scan the robot actually observes to the scan it would have hypothetically observed at each particle's position, computed by ray-casting. Ray-casting works by shooting beams from each particle and calculating the distance to the nearest obstacles.

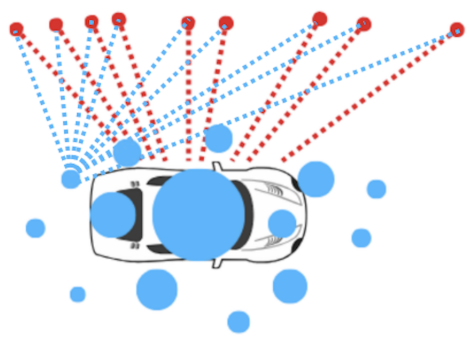


Figure 3: Each blue dot represents a particle, each blue line represents a ray-cast from the particle, and each red line represents a LiDAR beam. We compare the length of each blue line to the length of each red line to compute the likelihood of the blue dot occurring.

2.3.1 Precomputed Lookup Table

To achieve efficient real-time performance with many particles and beams, we precompute a 201×201 lookup table $p(z \mid d)$, where z is the discretized observed range (in pixels) and d is the discretized ray-cast range that we assume to be the ground-truth distance. This table is computed once at startup and indexed

at runtime.

The probability of a single beam measurement is a weighted sum of four components:

$$
\begin{aligned}
p(z \mid d) = \\
& \alpha_{\text{hit}}\, p_{\text{hit}}(z \mid d) \\
& + \alpha_{\text{short}}\, p_{\text{short}}(z \mid d) \\
& + \alpha_{\text{max}}\, p_{\text{max}}(z \mid d) \\
& + \alpha_{\text{rand}}\, p_{\text{rand}}(z \mid d) \\
& \text{s.t. } \alpha_{\text{hit}} + \alpha_{\text{short}} + \alpha_{\text{max}} + \alpha_{\text{rand}} = 1
\end{aligned} \tag{3}
$$

Each of the four probability models ($p_{hit}, p_{short}, p_{max}$, and p_{rand}), represent distinct physical phenomena and allow us to encode properties about the environment that the LiDAR detects:

1. **Hit** (p_{hit}): the probability that the beam strikes an expected map obstacle, producing a reading close to d with Gaussian noise $\sigma_{\text{hit}} = 10.0$ pixels. This is the dominant term: $\alpha_{\text{hit}} = 0.80$.

2. **Short** (p_{short}): the probability of an unexpected obstacle (e.g., a person or clutter) that causes a shorter-than-expected reading. Modeled as a linearly decreasing distribution over $[0, d]$: $\alpha_{\text{short}} = 0.07$.

3. **Max** (p_{max}): the probability of a large reading due to a highly reflective or transparent surface, modeled as a point mass at z_{max}: $\alpha_{\text{max}} = 0.07$.

4. **Random** (p_{rand}): the probability of complete randomness, modeled by a uniform distribution over $[0, 201]$: $\alpha_{\text{rand}} = 0.06$.

After computing the cumulative probability, or the product of each four probability models, each column of the table is normalized to sum to 1 (we want probability over each z_k to sum to 1).

2.3.2 Particle Weight Computation

At runtime, the sensor model recieves LiDAR scans and the current particle set. For each particle, it uses PyScanSimulator2D to ray-cast the expected scan from for that particle's pose on the map. The observed ranges and the ray-cast ranges are converted from meters to pixels using the formula

$$n_{pixels} = \frac{n_{meters}}{\text{map resolution} \times \text{lidar to map scale}}$$

and the updated pair z_k, d is used for look up in the precomputed table detailed in section **2.3.1**.

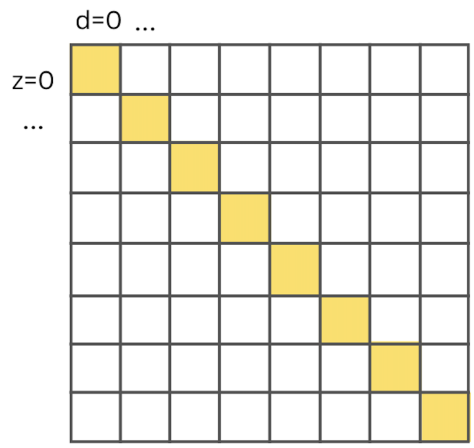


Figure 4: Our precomputed table is a matrix of probability values for observed distance z and ground truth distance d. In the table above that visualizes the $z = d$ diagonal, these values represent the probabilities that our observed distance is exactly equal to our real distance, or that LiDAR beams fit perfectly to our map.

If we assume that each ray-cast beam is independent, then the joint probability of the full scan per particle is the product of each beam's probabilities:

$$p(z_k \mid x_k, m) = \prod_{i=1}^{n} p\Big(z_k^{(i)} \mid x_k, m\Big) \tag{4}$$

To prevent numerical underflow from multiplying many small probabilities and to reduce the sensitivity to outlier beams, we raise each particle's weight to the power 0.7 (i.e., we flatten the weight distribution):

$$w^{(i)} \leftarrow \Big(w^{(i)}\Big)^{0.7} \tag{5}$$

This flattening was an important tuning choice and will be discussed further in section **3.2**: without it, a single cluttered wall could cause the weights to develop extreme peaks, effectively zeroing out all particles outside those peaks and causing filter collapse in high-clutter environments.

2.4 Particle Filter

2.4.1 Initialization

When a pose estimate is sent via RViz's "Pose Estimate" tool (publishing to the topic `/initialpose`), the particle filter initializes all N particles at that pose with small Gaussian perturbations ($\sigma = 0.5$ m, 0.5 rad).

2.4.2 Resampling

We use low-variance sampling (also known as systematic resampling), which draws a single random number $r \sim \text{Uniform}(0, 1/N)$ and places N evenly spaced pointers:

$$c_j = r + \frac{j-1}{N}, \quad j = 1, \ldots, N \tag{6}$$

Each pointer selects a particle from the cumulative weight distribution circularly. We used low-variance sampling as opposed to multinomial resampling, since this method reduces sampling variances and produces better particle diversity (prevents collapse).

After resampling, a small amount of Gaussian noise ($\sigma = 0.05$ m for position, $\sigma = 0.02$ rad for heading) is added to each resampled particle to model an appropriate spread with sensor noise.

2.4.3 Average Pose Estimate

To calculate our average pose estimate from our particle cloud, we take the average of the X and Y coordinates and a circular mean of the orientation angle. This average was chosen to take into account the circular nature of angles. We believe it is representative enough of the coordinates under the assumption that we are resampling from the weighted distribution.

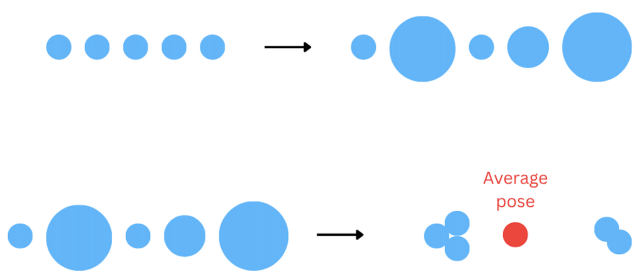


Figure 5: From top left to bottom right - start with our particle set. Then, apply weights found by the sensor model. Resample using low-variance sampling to come up with new particle poses. Finally, calculate the average pose from this new distribution.

3 Experimental Evaluation

3.1 Technical Procedures

Simulation evaluation.
In Simulation, we have access to ground truth about the car's location in the map, accurate LiDAR scans, and odometry. This allows us to compute the Pose cross-track error (CTE) which is the difference between the average pose estimate and the ground-truth pose of the robot. We've chosen this metric to provide insight as to the accuracy of our localization where low CTE is a successful measurement.

Physical evaluation.
We evaluated performance using two primary metrics:

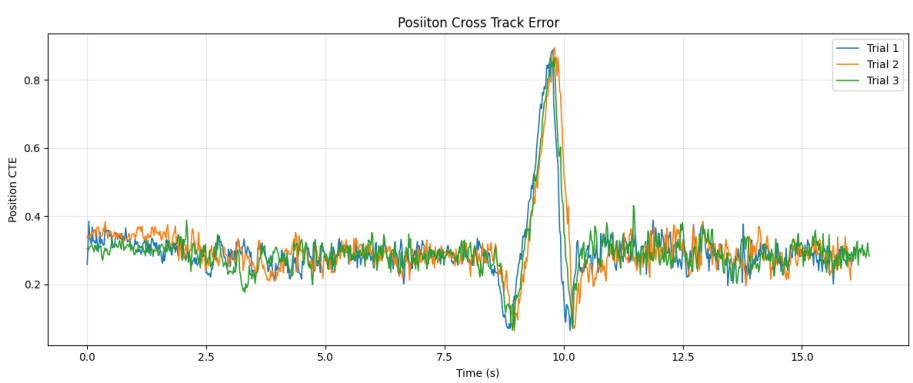
1. **Path convergence smoothness:** A qualitative analysis of the shape of the estimated path of the car where a smoother path corresponds to a more feasible one in the real world and a jagged path with stuttering is indicative of an uncertain localization model.

2. **Sensor model computation time:** The time it takes to generate new particles from receiving the LiDAR scan data.

3.2 Results

3.2.1 Simulation Results

We find that our particle filter is very accurate in simulation. Accuracy is measured using CTE, which yields low values between 0.2-0.4 meters for position and near zero values for orientation. We observed these low CTE values in the main hallway, bike hallway, and full loop around the Stata basement, showing that in simulation our particle filter is robust to changes in environment.



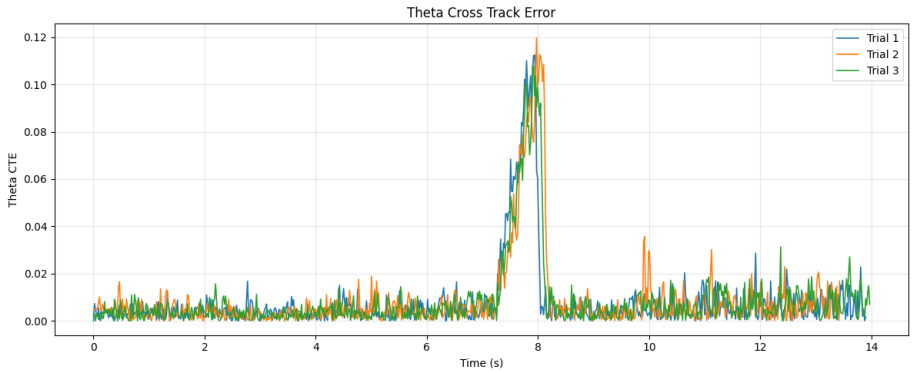


Figure 6: Cross-track error for position and orientation in simulation for the main hallway. The large spike in cross-track error occurs when there is a brief bend in the path with obstacles, causing uncertainty in the sensor model about the robot's current position. However, the graph shows that position error hovers between 0.2-0.4 meters and orientation error is almost negligible.

3.2.2 Real-World Results

Real-world Trial 1 (Main Hallway of Stata). Qualitative analysis of the localizer shows that it performs well in the Main Hallway of the Stata Basement. This hallway has many unique features that make it easier to localize in this section of the map. Additionally, conditions in the real world during the runs matched the map well which also contributes to good localization.

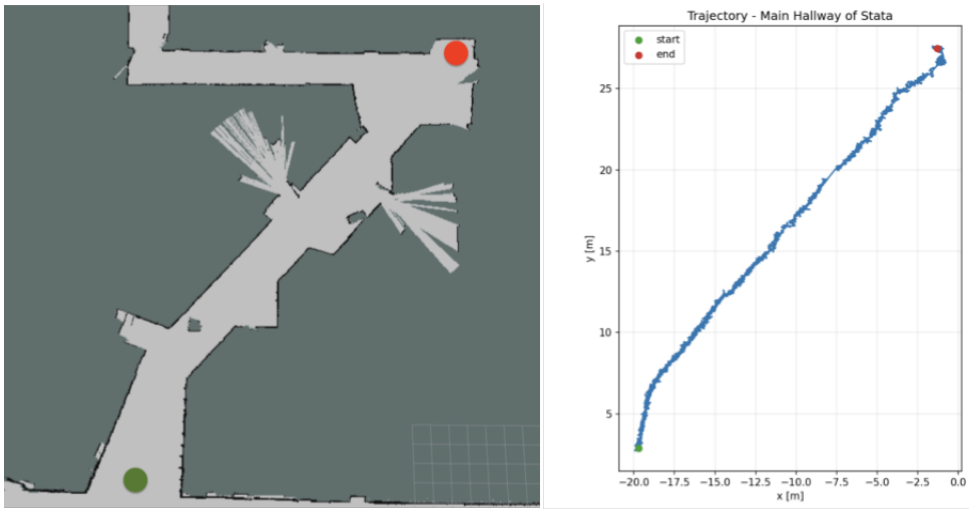


Figure 7: Plot of localized poses is relatively smooth and accurately matches the path taken through the map in the real world (based on comparison to video of the run).

Real-world Trial 2 (Bike Hallway).
When testing localization on another sector of the map, the resulting estimated path was not nearly as smooth. This sector of the map is different from the main hallway in that the map has less distinctive features and the map was not representative of the true obstacles observed in the real world. While the map showed flat walls, there was clutter lining the walls all the way down the hallway.

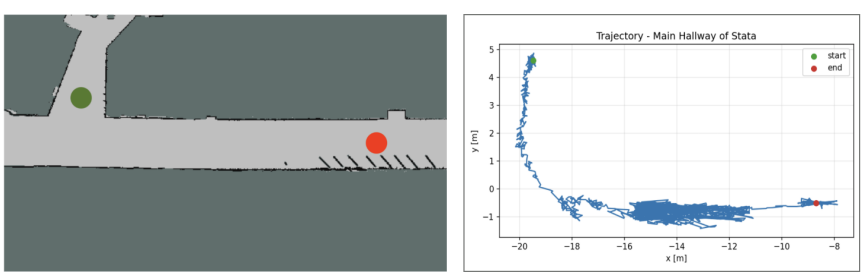


Figure 8: The first iteration of particle filtering model. Stutter can be observed from the interval $x \in [-16, -11]$.

Noise and Parameter Tuning to Prevent Weight Collapse

A common issue with localization is discrepancy between the real-world and assumed map layout. Unexpected clutter can lead to a phenomena known as particle/weight collapse, where the sensor model updates to resample only a handful of particles, while the other particles essentially disappear due to extremely low probabilities.

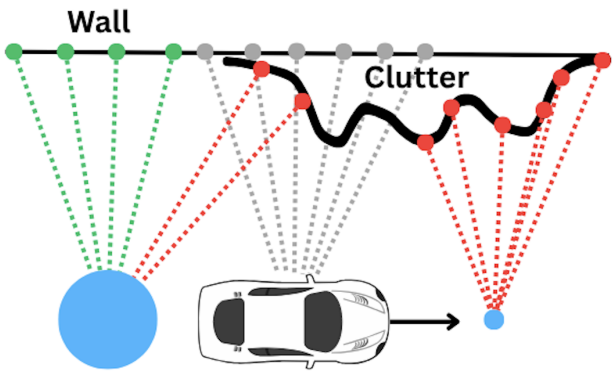


Figure 9: In the figure above, the car experiences clutter along the wall that the sensor model doesn't expect. The particle in front of the car ray-casts to the clutter, and p_{hit} becomes really low for each ray-cast beam since the map layout does not match the shape of the ray-cast. By contrast the particle behind the car is assigned higher weights because it sees the wall which the sensor model expects.

In practice, particle collapse can lead to poor performance - even if the car is moving forward in a cluttered environment, it may appear to "stutter," and the belief state fails to update forwards.

We observed particle collapse with the bike hallway which led to the non-smooth path shape (Figure 8). To address this issue, we (1) added noise to the motion model and (2) flattened weights in the sensor model.

By adding noise to the motion model, particles automatically become more spread out, even if they are updated by the sensor model to collapse to a single point as seen in Figure 10.

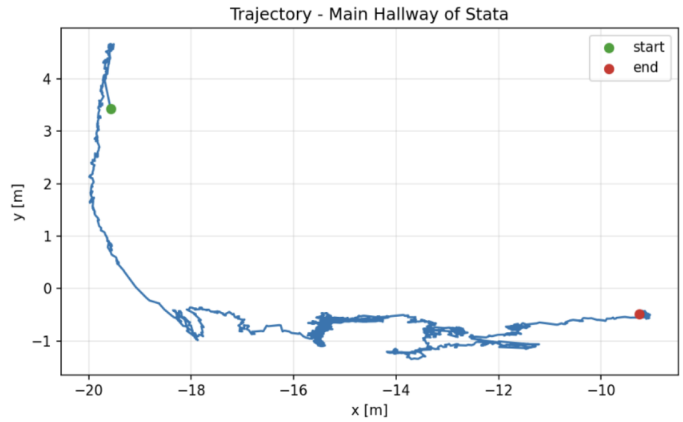


Figure 10: The second iteration of the model. Added noise fixes some of the stutter.

By flattening the weights of the sensor model, the weight distribution of particles naturally has less "peaks" that cluster around one point, allowing resampling to draw from a more diverse distribution of points.

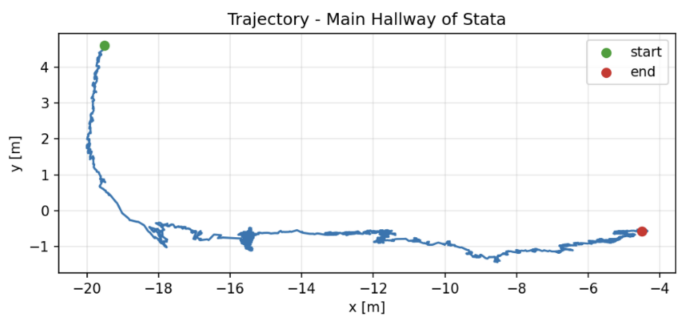


Figure 11: The third iteration of the model. Weight flattening and added noise work together to push the car past the original point of failure.

In both model modifications, the car has smoother path convergence down the hallway and stutters less. Indeed, the estimate endpoint in Figure 11 is closer to the ground truth endpoint of the car.

3.2.3 Computation Time

Similarly to how it is important that our map match the existing world for good localization, it is also important that our perception of the world be relevant

to where we currently are in the world. Updating the particles via the sensor model in a timely matter is an important way of maintaining this accuracy.

Across all of our different runs, the computation time of the sensor model was consistently under 60 microseconds. This is a rate of nearly 17 kHz. The LiDAR scans were published at a rate of 85 Hz. Relative to the rate at which scans are received, the time it takes to do the computation on them is sufficiently fast and gives us accurately updated particles.

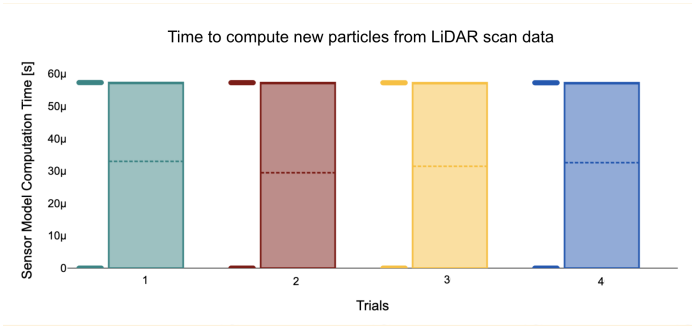


Figure 12: Consistently under 60 microseconds to compute new particles from scan data across multiple trials.

4 Conclusion

Using LiDAR and odometry, we are able to use the Monte Carlo Localization method to estimate the position of the car in a 2D map both in simulation and in real life. Our current implementation does well in sections of the map with distinctive features and where the map matches the real world obstacles. Refining this localization will be important as it will be a fundamental component to any further tasks such as path planning and navigating through the map according to that path. We begun refining by adjusting the noise we inject in the particle updates in the motion model and with weight flattening in the sensor model. We could also explore adjusting the 4 parameters used in the sensor model to compute the likelihood of a particle to account for the clutter that adds noise to our sensor data.

To evaluate the impact these changes have in real-world localization, we can also create a heuristic for our path convergence confidence. Ideally, this heuristic could be a quantitative representation of how sure we are that our localization is converging to an accurate path. It would likely be a function related to the shape and variance of the particle cloud. We would also like to measure the CTE relative to a real world path which would involve careful navigation so that we have measurements of the true location of the car in the map to compare to.

Making these adjustments will help us find the proper balance between relying on the motion model and adjustments of the sensor model to provide accurate, real-time localization.

5 Lessons Learned

Nina Garcia-Ruiz. Having rosbags to test with is super helpful when we want to do parameter testing and get more accurate analysis. It can also be useful to test computation time of our model as a means of understanding the accuracy of it. Also, it is important to agree about how pieces of the code conect. This way we can work in parallel. Also, this gives all team members a good base understanding of all parts of the lab.

Brendon Murphy: Thoroughly test your models in simulation first before moving on to real-world development and testing. We tested our simulation for the feature-rich main hallway and moved on to the real-world without ensuring our particle filter was robust to changes in environment. This led to a lot of issues during testing that set us back and delayed completion of the lab. Testing everything in simulation first ensures implementation is sound, and makes it easier to fix changes rather than debug in real life

Irene Tang: Understanding how each part of the model works individually is helpful in debugging the whole system when something breaks - it allows you to narrow down failure points or bottlenecks more accurately then having to guess-and-check every single time. Knowing which parameters to change, how and where, can save you a lot of time in the long-run.

Vandell Vatel: Maintaining open lines of communication is highly important when work has been divided to ensure everyone is familiar with all parts of the lab.